WHAT IS DISTINCT

DISTINCT removes **duplicate rows** from the result.

SELECT DISTINCT column_name FROM table;

👉 Think:

"Give me only **unique values** from this result set"

🔍 **Core Concept (Most Important)**

👉 DISTINCT works on **entire row combinations**, not individual columns.

📌 **Example Table**

name	course
Indra	SQL
Indra	SQL
Indra	Python
Ravi	SQL

Case 1: Single Column

SELECT DISTINCT name FROM student_courses;

👉 Output:

Indra

Ravi

Case 2: Multiple Columns

SELECT DISTINCT name, course FROM student_courses;

👉 Output:

Indra SQL

Indra Python

Ravi SQL

👉 Here uniqueness = **(name + course combination)**

⚙️ How It Works Internally

MySQL:

1. Reads rows
2. Compares values
3. Removes duplicates (using sorting or hashing)

👉 That's why:

- It can be **slow on large data**
- Needs **extra memory**

🚀 Real Use Cases

1. Get Unique Values

SELECT DISTINCT country FROM customers;

👉 "Which countries do we have users from?"

---

2. Remove Duplicate Data in Reports

SELECT DISTINCT email FROM users;

---

3. Unique Combinations

SELECT DISTINCT city, country FROM customers;

---

4. Count Unique Values

SELECT COUNT(DISTINCT country) FROM customers;

👉 Very common in interviews

⚠️ Common Mistakes (Very Important)

❌ Mistake 1: Thinking it works per column

SELECT DISTINCT name, course

👉 ❌ Not separate uniqueness
👉 ✅ Works on combination

❌ Mistake 2: Using with ORDER BY incorrectly

SELECT DISTINCT name FROM students ORDER BY age;

👉 ❌ Error (like you saw)

✓ Fix:

SELECT DISTINCT name, age FROM students ORDER BY age;

❌ Mistake 3: Using DISTINCT instead of fixing bad data

👉 If duplicates exist in table:

- Fix using constraints (UNIQUE)
- Don't rely on DISTINCT every time

❌ Mistake 4: Overusing DISTINCT

SELECT DISTINCT * FROM big_table;

👉 🚫 Slow + unnecessary

🆚 DISTINCT vs GROUP BY

Same Result (sometimes)

SELECT DISTINCT country FROM customers;

SELECT country FROM customers GROUP BY country;

👉 Output same

But GROUP BY is more powerful

SELECT country, COUNT(*)

FROM customers

GROUP BY country;

👉 DISTINCT cannot do this

🔥 Interview-Level Insights

💡 Q1: When to use DISTINCT?

- When you only need **unique values**

- No aggregation required

💡 **Q2: DISTINCT vs GROUP BY?**

DISTINCT	GROUP BY
Removes duplicates	Groups data
Simple	More powerful
No aggregation	Supports aggregation

💡 **Q3: Performance?**

- DISTINCT = sorting / hashing
- Costly on large tables
- Index helps a lot

💡 **Q4: Multiple DISTINCT?**

COUNT(DISTINCT col1, col2)

👉 Works in MySQL (combination uniqueness)

🧪 **Practice Scenarios**

1. Unique job roles

SELECT DISTINCT job_title FROM employees;

---

2. Unique city-country pairs

SELECT DISTINCT city, country FROM customers;

---

3. Count unique users

SELECT COUNT(DISTINCT user_id) FROM orders;

🧠 **Mental Model (Remember This Forever)**

👉 DISTINCT =

"Remove duplicate **rows**, not columns"

⚡ **Pro Tip (Real World)**

If you see yourself using DISTINCT a lot:

👉 Ask:

- Why duplicates exist?
- Can I fix schema instead?

🎯 The Rule (Simple Version)

When you use DISTINCT, you can only ORDER BY columns that are in the SELECT list.

🧠 **Why This Rule Exists (Intuition First)**

DISTINCT **removes duplicate rows**.

After duplicates are removed, some column values are **lost**.

👉 So if you try to sort using a column that's not selected, SQL asks:

"Which value should I use for sorting? I don't have it anymore."

🔍 **Clear Example**

📌 **Table**

source_of_joining	years_of_exp
Referral	5
Referral	10
LinkedIn	3

❌ **Your Query**

```
SELECT DISTINCT source_of_joining
FROM students
```

ORDER BY years_of_exp DESC;

🚨 **What Happens Internally**

Step 1: Apply DISTINCT

source_of_joining

Referral

LinkedIn

👉 Now years_of_exp is gone ❌

Step 2: Try to ORDER

ORDER BY years_of_exp DESC;

👉 SQL gets stuck:

- For Referral, should it use **5 or 10**?
- It doesn't know → ❌ ERROR

✅ **Fix 1 (Add column to SELECT)**

SELECT DISTINCT source_of_joining, years_of_exp

FROM students

ORDER BY years_of_exp DESC;

👉 Now SQL can sort, because data is present.

⚠️ But now DISTINCT works on **both columns**, not just one.

✅ **Fix 2 (Correct Logic using GROUP BY)**

If your goal is:

"Sort unique sources by highest experience"

Then write clearly:

SELECT source_of_joining, MAX(years_of_exp) AS max_exp

FROM students

GROUP BY source_of_joining

ORDER BY max_exp DESC;

👉 Now:

- No confusion
- Correct logic
- Interview-level answer

🧠 **Simple Analogy**

Think like this:

You say:

"Give me unique names"

Then you say:

"Sort them by salary"

👉 But salary is not selected!

So SQL asks:

"Which salary?? You removed that info."

🔥 **Key Takeaway**

- DISTINCT removes rows → removes data
- ORDER BY needs data to sort
- If data is missing → ❌ error

🚀 **Straight Truth**

Your query wasn't just syntactically wrong—it was **logically incomplete**.

And SQL is strict about logic.

📌 Example -2 Table: students

name	marks
A	90
A	80

name	marks
B	70

❌ Query (your type of mistake)

```
SELECT DISTINCT name
FROM students
ORDER BY marks DESC;
```

🧠 What happens step-by-step

Step 1: Apply DISTINCT

```
SELECT DISTINCT name
```

👉 Result becomes:

name
A
B

Step 2: Try to sort

```
ORDER BY marks DESC;
```

👉 Problem:

- For **A**, marks = 90 and 80 🤯
- Which one should SQL use?

👉 SQL says:

"I don't know → ERROR"

✅ Fix 1 (Add column)

```
SELECT DISTINCT name, marks
FROM students
ORDER BY marks DESC;
```

👉 Now result:

name	marks
A	90
A	80
B	70

✓ No confusion
⚠️ But duplicates of name still exist

✅ **Fix 2 (Correct logic)**

👉 If you want:

unique students sorted by highest marks

```
SELECT name, MAX(marks) AS max_marks
FROM students
GROUP BY name
ORDER BY max_marks DESC;
```

👉 Result:

name	max_marks
A	90
B	70

✓ Clean
✓ Logical
✓ Interview-ready

🧠 **One-line Memory Trick**

If you remove a column using DISTINCT, you cannot use it for sorting.